

Agente de IA Minimalista com Análise Temporal

Relatório da atividade: nova tool, novo problema, testes e troca de LLM

Aluno	Nickolas Corazza Alves (RM 562265)
Disciplina	Cloud Artificial Intelligence
Professor	Fernando Nemec
Apresentação	02/10/2026
Ambiente	Windows 11, VS Code, Python 3.11.9, notebook Jupyter; API Gemini no plano gratuito

1. Resumo

Parti do notebook do professor (agente investigador de incidentes de nuvem, com 7 ferramentas, usando a API da OpenAI) e fiz os quatro itens pedidos: (1) criei a ferramenta **analisar_serie_temporal**; (1.1) criei dois problemas novos, com séries de métricas ao longo do tempo; (2) montei um avaliador automático que repete cada cenário, com e sem a ferramenta, e confere por código se o agente pediu o rollback correto; (3) portei o agente para o Google Gemini, no plano gratuito. Todos os números deste relatório vêm das execuções reais no meu computador.

Principais resultados:

- A ferramenta funciona sozinha e é usada pelo agente. Ela localiza o início de uma mudança de tendência (memória do serviço de catálogo: 12:05, com margem de 5 minutos) e mostra o deploy do mesmo instante (dep-451, 12:10).
- No cenário original, o agente acertou o rollback em **12 de 12** execuções nas quatro últimas avaliações (modelo gemini-3.1-flash-lite).
- Nos problemas novos, o resultado foi diferente do esperado. No cenário C (o mais difícil), o modelo pequeno acertou **0 de 9** com a ferramenta e **0 de 9** sem ela. O diagnóstico dos relatórios mostrou o motivo: o agente confundia o sintoma (latência) com a causa e culpava o deploy mais próximo dele.
- Com um modelo maior (gemini-3.5-flash), o agente pediu o rollback correto nas **2 de 2** execuções feitas. Mas são só 2 execuções, a cota gratuita (20 requisições por dia) impediu repetir, e não medi o mesmo modelo sem a ferramenta. Portanto **não afirmo** que a ferramenta foi o motivo do acerto.

2. Ponto de partida: o notebook do professor

O agente recebe um alerta, escolhe uma ferramenta, lê o resultado e repete até escrever um relatório (máximo de 8 rodadas, 350 tokens de saída por rodada, teto de custo de US\$ 0,02). As 7 ferramentas são: métricas, logs, deploys, configuração, custos, eventos recentes e solicitar rollback (que apenas abre um pedido; a aprovação é humana). O incidente original (INC-2026-0911): o deploy dep-442, às 13:58, configurou o recommendation-api para chamar a si mesmo como fallback, criando um loop. A CPU foi a 98%, as instâncias de 2 para 48 e o custo de US\$ 0,31 para US\$ 7,44 por hora (24 vezes).

Limitação observada: as métricas do mockup são uma foto de um único instante e os eventos são uma lista pronta de 4 itens. Não existe como perguntar "quando isso começou?". Por isso o cenário original não exige investigação temporal.

3. O que foi feito

3.1 Nova ferramenta: analisar_serie_temporal (item 1)

Recebe um serviço e, opcionalmente, uma métrica (CPU, memória, latência p95 ou taxa de erro). Como funciona, em termos simples:

- Testa cada horário como o possível "momento da quebra": antes dele a série é uma linha plana, depois dele é uma reta inclinada. O horário em que esse modelo melhor encaixa nos dados é o **ponto de mudança**.
- Só declara mudança se o encaixe for bom e a variação for grande em relação ao ruído normal. Séries saudáveis saem como "estáveis".
- Lista os deploys e eventos até 30 minutos antes ou depois do ponto de mudança, dizendo se vieram **antes, depois ou no mesmo instante** (margem de erro de 5 minutos).
- **Evolução após os testes:** na versão 3.1, sem informar a métrica, a ferramenta faz uma **varredura** e mostra a ordem em que as métricas do serviço mudaram, e avisa quando outra métrica mudou antes da analisada. Na versão 3.2, ela checka se um deploy posterior **alterou a inclinação** da métrica de origem.

Exemplo real (saída do teste da ferramenta sem IA, cenário C, memória do catalog-api):

```
mudanca_de_tendencia: 12:05 | baseline_antes_da_mudanca: 54.81 | valor_atual: 91.53 |  
inclinacao apos mudanca por hora: 8.77 | eventos_proximos: dep-451 (deploy, 12:10), diferenca_min:  
5, posicao: no mesmo instante (dentro da margem de erro)
```

3.2 Problemas novos no mockup (item 1.1)

Adicionei ao mockup séries de 11:00 a 16:15 (5 em 5 minutos) geradas com semente fixa, ou seja, os números são sempre os mesmos. A lista de eventos recentes mostra apenas os últimos 90 minutos, como um painel real.

Cenário	Causa real (gabarito)	Pistas falsas	Observação
A - original	dep-442 (13:58): fallback recursivo	-	Não exige análise temporal (foto de um instante). Mantido para provar que nada quebrou.
B - vazamento de memória (INC-2026-0912)	dep-451 (12:10): cache de itens sem expiração; a memória do catalog-api cresce 8,8 pontos por hora; a latência só dispara às 15:30 e o checkout piora às 15:40	dep-455 (15:52, checkout): só um banner. Veio DEPOIS do sintoma	Falha de projeto minha: a descrição do deploy dizia cache_ttl_s 300 para 0 e entregava a resposta.
C - deploys confusos (INC-2026-0913)	dep-451 (12:10): "refatoração do módulo de listagem" (sem pista textual)	dep-453 (15:25): log em DEBUG; dep-455 (15:52)	Criado depois que a avaliação mostrou o B fácil demais. As séries são as mesmas do B.

3.3 Testes e avaliação (item 2)

Montei um avaliador automático. Cada caso roda 3 vezes (o modelo não é determinístico) e o código confere: **Rollback** = pediu o rollback do deploy correto e nunca o errado; **Concluiu** = terminou com relatório; **Menciona** = o relatório cita o ID do deploy correto (métrica fraca: só detecta o texto, não quem o relatório culpa). Compara o agente **com** e **sem** a ferramenta temporal (ablação) e registra rodadas, tokens, custo equivalente, o motivo de cada falha e o texto do relatório. Os resultados são salvos a cada execução e retomados se algo parar.

3.4 Troca de LLM (item 3)

O notebook original usa o modelo gpt-5.6-luna da OpenAI (conferi na documentação: US\$ 0,20 por milhão de tokens de entrada e US\$ 1,20 de saída, os mesmos valores do notebook). Portei o agente para o **Google Gemini**, pela API nativa e sem bibliotecas extras. Motivo: o plano gratuito não exige cartão, o que faz sentido para um aluno. Mudanças técnicas: formato das ferramentas (functionDeclarations), mensagens (functionCall e functionResponse), preservação da thoughtSignature exigida pelo Gemini 3, e nova tentativa automática em erros 503, 429 e timeout. **Não rodei o agente na OpenAI** (não tenho chave), então não comparo a qualidade entre as duas empresas. O "custo equivalente" usa os tokens

reais de cada execução e o preço do GPT-5.6 Luna; o custo real no plano gratuito é zero.

4. Resultados

4.1 Modelo gemini-3.1-flash-lite (39 execuções avaliadas automaticamente)

Valores: acertos de rollback / execuções | rodadas médias | tokens médios | custo equivalente médio em US\$. O custo real foi zero.

Avaliação (versão)	A original, com tool	B, sem tool	B, com tool	C, sem tool	C, com tool
1a (v2.2)	2/3 5,3 14.180 0,00454	2/3 5,7 9.653 0,00300	1/3 6,7 19.529 0,00548	-	-
v3	3/3 4,0 10.491 0,00328	-	-	0/3 8,0 16.076 0,00471	0/3 5,0 14.716 0,00431
v3.1	3/3 4,7 12.966 0,00397	-	-	0/3 5,3 10.928 0,00350	0/3 6,3 19.938 0,00605
v3.2	3/3 5,7 17.554 0,00529	-	-	0/3 5,3 11.345 0,00375	0/3 6,0 18.494 0,00504
v4.1 (só A)	3/3 4,7 15.355 0,00479	-	-	-	-

4.2 Modelo gemini-3.5-flash (teste manual, cenário C, com a ferramenta)

Execução	Rollback pedido	Concluiu com relatório	Rodadas	Tokens
#1	dep-451 (correto)	Sim	8	29.432
#2	dep-451 (correto)	Não (estourou as rodadas)	8	24.469

A avaliação seguinte desse modelo foi interrompida: ele tem limite de 20 requisições por dia no plano gratuito (mensagem de erro 429 do Google) e a conta já estava no limite. Média de tokens: 26.950, cerca de 46% a mais que o modelo leve com a ferramenta (18.494 na v3.2).

4.3 O que os resultados mostram

- **Cenário A estável:** 12 de 12 nas avaliações v3, v3.1, v3.2 e v4.1. O custo e as rodadas variaram entre avaliações (US\$ 0,00328 a 0,00529; 4,0 a 5,7 rodadas) sem mudança no cenário, então a variação é ruído e **não posso afirmar que alguma versão ficou mais barata**.
- **Cenário B:** a ferramenta não ajudou (com tool 1/3, sem tool 2/3), com custo 1,8 vez maior e o dobro de tokens. Nenhuma das 6 execuções do cenário B pediu o rollback da pista falsa (dep-455). Conclusão: o B era fácil demais, porque o texto do deploy dava a resposta.
- **Cenário C com o modelo pequeno:** 0 de 9 com a ferramenta e 0 de 9 sem. Nas execuções cujo detalhe vi, o rollback pedido foi sempre o do dep-453 (a pista falsa).
- **Uso da ferramenta:** nas avaliações mais recentes (v3 em diante), o agente a usou em todas as execuções em que ela estava disponível.

4.4 Diagnóstico das falhas no cenário C

Li os relatórios do agente e encontrei dois erros de raciocínio diferentes:

- **Execução que olhou só o sintoma:** o agente usou a ferramenta uma vez, no checkout-api (o serviço do alarme), nunca analisou a memória do catalog-api e culpou o dep-453 por ele ter vindo logo antes da piora da latência.
- **Execução que viu a origem e decidiu errado:** o agente viu que a memória do catalog-api crescia desde 12:05, após o dep-451, mas escreveu que o problema "agravou-se drasticamente" depois do dep-453 e pediu o rollback dele. Nenhum dado sustentava isso: a memória cresce em ritmo constante, sem mudança às 15:25.

- **Correções tentadas:** varredura e aviso de "métrica que mudou antes" (v3.1) e checagem de inclinação mais regra de decisão nas instruções (v3.2). Nenhuma mudou o resultado do modelo pequeno (0 de 3 em cada avaliação).
- **Erro na minha métrica:** o "Menciona" deu 2 de 3 na v3.1, mas em uma dessas execuções o relatório citava o dep-451 e mesmo assim pedia o rollback do dep-453. Por isso a métrica que vale é o rollback pedido.

5. Inovação e evolução em relação ao agente original

Cada melhoria abaixo nasceu de um problema observado nos testes. Indico também o que **não** se mostrou eficaz.

Melhoria	Motivo (dado observado)	Situação
Nova tentativa automática (503, 429, timeout)	Erros 503 em quase toda execução; um timeout derrubou uma avaliação inteira; 78 retentativas registradas nas avaliações	Funcionou: avaliações seguintes sem execuções perdidas por infraestrutura
Guarda de qualidade do relatório	Um relatório do cenário A veio com uma frase só	Vi a guarda agir em 1 execução (relatório de 115 caracteres) e o relatório final saiu completo. Nas execuções automáticas ela é silenciosa, então não sei quantas vezes agiu
Última rodada reservada ao relatório e aviso de orçamento	Execuções estouravam as rodadas sem relatório	Parcial: 3 execuções que chegaram a 8 rodadas (C sem tool, v3) terminaram com relatório, mas 2 ainda estouraram o limite depois da melhoria (uma no cenário B e uma no gemini-3.5-flash)
Varredura temporal, aviso de métrica anterior e checagem de inclinação	O agente analisava só o sintoma (latência)	Ferramenta funciona isolada; sem efeito no modelo pequeno
Avaliador automático com ablação (com/sem tool), motivos de falha, salvamento e retomada	Precisava medir, não só observar uma execução	Funcionou: revelou que o cenário B era fácil e expôs as falhas do C
Custo equivalente por execução	Plano gratuito esconde o custo real	Funcionou: fração de centavo por execução
Parada rápida na cota diária e troca temporária de modelo	Erro real: 20 requisições por dia no gemini-3.5-flash; minha detecção inicial falhou porque cortava a mensagem do Google	Corrigido e testado com uma réplica da mensagem real
Eliminação de chamadas repetidas	Economia de tokens	Contador em 0,0 nas avaliações v3, v3.1 e v3.2 (27 execuções): sem efeito medido
Instrução para chamadas em paralelo	Gastar menos requisições da cota	Efeito ainda não medido

6. Limitações (o que este trabalho não prova)

- **Amostras pequenas:** 3 execuções por caso (2 no modelo maior). Serve como indício, não como prova estatística.
- **Gabarito definido por mim:** a causa "correta" é a que a simulação define. O dep-453 (log em DEBUG) tem um mecanismo plausível para piorar latência, então a escolha do agente é discutível, e não absurda.
- **Dados fictícios** gerados com semente fixa; não são medições de um ambiente real.
- **O efeito da ferramenta no modelo maior não foi isolado:** falta rodar o gemini-3.5-flash sem a ferramenta no cenário C.

- **Comparações entre versões são indicativas:** as instruções mudaram entre as avaliações, e os erros 503 e as novas tentativas podem influenciar o comportamento.
- **Sem comparação direta com a OpenAI** (sem chave).

7. Conclusão e próximos passos

A ferramenta temporal é tecnicamente correta e o agente a utiliza, mas o ganho de resposta depende da capacidade do modelo: o modelo pequeno confunde sintoma com causa mesmo com a evidência certa na mão. Os testes, mais do que confirmar a ideia, mostraram onde ela falha e permitiram diagnosticar o motivo. Próximos passos: (1) rodar o gemini-3.5-flash com e sem a ferramenta, com mais execuções (em dias diferentes, por causa da cota, ou em um plano com cota maior); (2) testar o gemini-3.5-flash-lite, que tem 500 requisições por dia e ainda não foi testado; (3) repetir com mais cenários e sementes diferentes.

Anexo A - Limites da minha conta (painel Rate limit do Google AI Studio, plano gratuito)

Modelo	Requisições por minuto	Requisições por dia	Uso registrado
Gemini 3.5 Flash	5	20	23 / 20
Gemini 3.1 Flash Lite	15	500	309 / 500
Gemini 3.5 Flash Lite	15	500	0 / 500
Gemini 3.8 Flash	5	20	1 / 20
Gemini 3.6 Flash e 3.7 Flash	5	20	0 / 20
Gemma 4 26B e 31B	30	14.400	0 / 14.4K

A documentação oficial do Google informa que as cotas diárias são por projeto e zeram à meia-noite do horário do Pacífico. Não testei se os modelos Gemma aceitam chamada de ferramentas.

Anexo B - Arquivos e como reproduzir

- **Agente_Gemini_v4_1.ipynb**: notebook final, com as saídas das execuções.
- **Relatorio_Agente_IA_Analise_Temporal.pdf**: este relatório.
- Para reproduzir: colocar a chave da API em um arquivo local gemini_key.txt (**nunca entregar esse arquivo**), abrir o notebook no VS Code, escolher o kernel Python 3.11 e rodar as células em ordem. A célula "Teste da nova tool" não usa a API e roda na hora.

Anexo C - Evidências (prints) a anexar

#	O que mostrar	Onde está
1	Teste da nova tool sem IA (varredura do catalog-api: memória às 12:05 com o dep-451)	Notebook, célula "Teste da nova tool SEM usar a IA"
2	Execução completa do cenário original: rodadas, tools, rollback do dep-442 aguardando aprovação	Notebook, "Execução 2: cenário A"
3	Tabela de avaliação do cenário A (3/3) com o modelo leve	Notebook, "Regressão do cenário A"
4	Mensagem de cota esgotada do gemini-3.5-flash (limit: 20)	Agente_Gemini_v4.ipynb, erro da Execução 1 (no v4.1 aparece só o aviso de cota esgotada)
5	Painel de limites da conta	aistudio.google.com, página Rate limit